

We categorize questions by *question type*, defined by the outermost function in the question’s program; for example the questions in Figure 2 have types *query-color* and *exist*. Figure 3 shows the number of questions of each type.

Question families. We must overcome several key challenges to generate a VQA dataset using functional programs. Functional building blocks can be used to construct an infinite number of possible functional programs, and we must decide which program structures to consider. We also need a method for converting functional programs to natural language in a way that minimizes question-conditional bias. We solve these problems using *question families*.

A question family contains a template for constructing functional programs and several text templates providing multiple ways of expressing these programs in natural language. For example, the question “How many red things are there?” can be formed by instantiating the text template “How many <C> <M> things are there?”, binding the parameters <C> and <M> (with types “color” and “material”) to the values *red* and *nil*. The functional program `count(filter_color(red, scene()))` for this question can be formed by instantiating the associated program template

`count(filter_color(<C>, filter_material(<M>, scene())))`

with the same values, using the convention that functions taking a *nil* input are removed after instantiation.

CLEVR contains a total of 90 question families, each with a single program template and an average of four text templates. Text templates were generated by manually writing one or two templates per family and then crowdsourcing question rewrites. To further increase language diversity we use a set of synonyms for each shape, color, and material. With up to 19 parameters per template, a small number of families can generate a huge number of unique questions; Figure 3 shows that of the nearly one million questions in CLEVR, more than 853k are unique. CLEVR can easily be extended by adding new question families.

Question generation. Generating a question for an image is conceptually simple: we choose a question family, select values for each of its template parameters, execute the resulting program on the image’s scene graph to find the answer, and use one of the text templates from the question family to generate the final natural-language question.

However, many combinations of values give rise to questions which are either *ill-posed* or *degenerate*. The question “What color is the cube to the right of the sphere?” would be *ill-posed* if there were many cubes right of the sphere, or *degenerate* if there were only one cube in the scene since the reference to the sphere would then be unnecessary. Avoiding such ill-posed and degenerate questions is critical to ensure the correctness and complexity of our questions.

A naïve solution is to randomly sample combinations of values and reject those which lead to ill-posed or degenerate

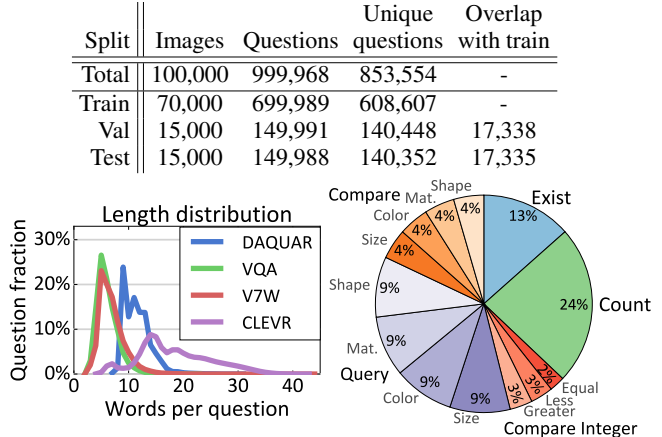


Figure 3. **Top:** Statistics for CLEVR; the majority of questions are unique and few questions from the val and test sets appear in the training set. **Bottom left:** Comparison of question lengths for different VQA datasets; CLEVR questions are generally much longer. **Bottom right:** Distribution of question types in CLEVR.

questions. However, the number of possible configurations for a question family is exponential in its number of parameters, and most of them are undesirable. This makes brute-force search intractable for our complex question families.

Instead, we employ a depth-first search to find valid values for instantiating question families. At each step of the search, we use ground-truth scene information to prune large swaths of the search space which are guaranteed to produce undesirable questions; for example we need not entertain questions of the form “What color is the <S> to the <R> of the sphere” for scenes that do not contain spheres.

Finally, we use rejection sampling to produce an approximately uniform answer distribution for each question family; this helps minimize question-conditional bias since all questions from the same family share linguistic structure.

4. VQA Systems on CLEVR

4.1. Models

VQA models typically represent images with features from pretrained CNNs and use word embeddings or recurrent networks to represent questions and/or answers. Models may train recurrent networks for answer generation [10, 28, 41], multiclass classifiers over common answers [4, 24, 25, 32, 48, 49], or binary classifiers on image-question-answer triples [9, 16, 33]. Many methods incorporate attention over the image [9, 33, 44, 49, 43] or question [24]. Some methods incorporate memory [42] or dynamic network architectures [2, 3].

Experimenting with all methods is logistically challenging, so we reproduced a representative subset of methods: baselines that do not look at the image (Q-type mode, LSTM), a simple baseline (CNN+BoW) that performs near state-of-the-art [16, 48], and more sophisticated methods

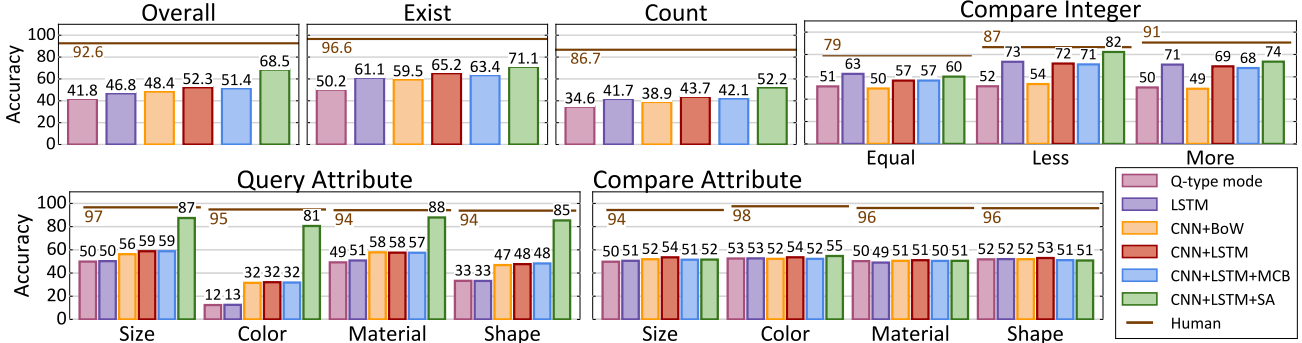


Figure 4. Accuracy per question type of the six VQA methods on the CLEVR dataset (higher is better). Figure best viewed in color.

using recurrent networks (CNN+LSTM), sophisticated feature pooling (CNN+LSTM+MCB), and spatial attention (CNN+LSTM+SA).¹ These are described in detail below.

Q-type mode: Similar to the “per Q-type prior” method in [4], this baseline predicts the most frequent training-set answer for each question’s type.

LSTM: Similar to “LSTM Q” in [4], the question is processed with learned word embeddings followed by a word-level LSTM [15]. The final LSTM hidden state is passed to a multi-layer perceptron (MLP) that predicts a distribution over answers. This method uses no image information so it can only model question-conditional bias.

CNN+BoW: Following [48], the question is encoded by averaging word vectors for each word in the question and the image is encoded using features from a convolutional network (CNN). The question and image features are concatenated and passed to a MLP which predicts a distribution over answers. We use word vectors trained on the Google-News corpus [29]; these are not fine-tuned during training.

CNN+LSTM: As above, images and questions are encoded using CNN features and final LSTM hidden states, respectively. These features are concatenated and passed to an MLP that predicts an answer distribution.

CNN+LSTM+MCB: Images and questions are encoded as above, but instead of concatenation, their features are pooled using compact multimodal pooling (MCB) [9, 11].

CNN+LSTM+SA: Again, the question and image are encoded using a CNN and LSTM, respectively. Following [44], these representations are combined using one or more rounds of soft spatial attention and the final answer distribution is predicted with an MLP.

Human: We used Mechanical Turk to collect human responses for 5500 random questions from the test set, taking a majority vote among three workers for each question.

Implementation details. Our CNNs are ResNet-101 models pretrained on ImageNet [14] that are not finetuned; images are resized to 224×224 prior to feature extraction.

¹We performed initial experiments with dynamic module networks [2] but its parsing heuristics did not generalize to the complex questions in CLEVR so it did not work out-of-the-box; see supplementary material.

CNN+LSTM+SA extracts features from the last layer of the conv4 stage, giving $14 \times 14 \times 1024$ -dimensional features. All other methods extract features from the final average pooling layer, giving 2048-dimensional features. LSTMs use one or two layers with 512 or 1024 units per layer. MLPs use ReLU functions and dropout [34]; they have one or two hidden layers with between 1024 and 8192 units per layer. All models are trained using Adam [20].

Experimental protocol. CLEVR is split into train, validation, and test sets (see Figure 3). We tuned hyperparameters (learning rate, dropout, word vector size, number and size of LSTM and MLP layers) independently per model based on the validation error. All experiments were designed on the validation set; after finalizing the design we ran each model once on the test set. *All experimental findings generalized from the validation set to the test set.*

4.2. Analysis by Question Type

We can use the program representation of questions to analyze model performance on different forms of reasoning. We first evaluate performance on each question type, defined as the outermost function in the program. Figure 4 shows results and detailed findings are discussed below.

Querying attributes: Query questions ask about an attribute of a particular object (e.g. “What color is the thing right of the red sphere?”). The CLEVR world has two sizes, eight colors, two materials, and three shapes. On questions asking about these different attributes, Q-type mode and LSTM obtain accuracies close to 50%, 12.5%, 50%, and 33.3% respectively, showing that the dataset has minimal question-conditional bias for these questions. CNN+LSTM+SA substantially outperforms all other models on these questions; its attention mechanism may help it focus on the target object and identify its attributes.

Comparing attributes: Attribute comparison questions ask whether two objects have the same value for some attribute (e.g. “Is the cube the same size as the sphere?”). The only valid answers are “yes” and “no”. Q-Type mode and LSTM achieve accuracies close to 50%, confirming there is no dataset bias for these questions. Unlike attribute-query

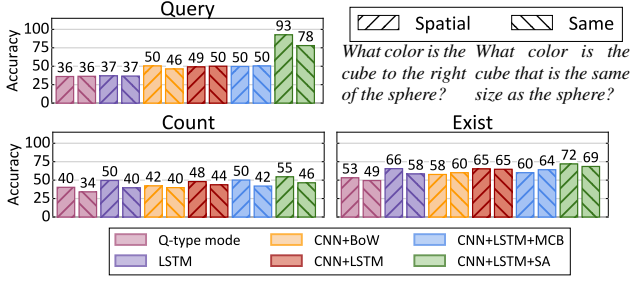


Figure 5. Accuracy on questions with a single *spatial relationship* vs. a single *same-attribute* relationship. For *query* and *count* questions, models generally perform worse on questions with *same-attribute* relationships. Results on *exist* questions are mixed.

questions, attribute-comparison questions require a limited form of memory: models must identify the attributes of two objects and keep them in memory to compare them. Interestingly, none of the models are able to do so: all models have an accuracy of approximately 50%. This is also true for the CNN+LSTM+SA model, suggesting that its attention mechanism is not capable of attending to two objects at once to compare them. This illustrates how CLEVR can reveal limitations of models and motivate follow-up research, e.g., augmenting attention models with explicit memory.

Existence: Existence questions ask whether a certain type of object is present (e.g., “Are there any cubes to the right of the red thing?”). The 50% accuracy of Q-Type mode shows that both answers are *a priori* equally likely, but the LSTM result of 60% does suggest a question-conditional bias. There may be correlations between question length and answer: questions with more filtering operations (e.g., “large red cube” vs. “red cube”) may be more likely to have “no” as the answer. Such biases may be present even with uniform answer distributions per question family, since questions from the same family may have different numbers of filtering functions. CNN+LSTM(+SA) outperforms LSTM, but its performance is still quite low.

Counting: Counting questions ask for the number of objects fulfilling some conditions (e.g. “How many red cubes are there?”); valid answers range from zero to ten. Images have three and ten objects and counting questions refer to subsets of objects, so ensuring a uniform answer distribution is very challenging; our rejection sampler therefore pushes towards a uniform distribution for these questions rather than enforcing it as a hard constraint. This results in a question-conditional bias, reflected in the 35% and 42% accuracies achieved by Q-type mode and LSTM. CNN+LSTM(+MCB) performs on par with LSTM, suggesting that CNN features contain little information relevant to counting. CNN+LSTM+SA performs slightly better, but at 52% its absolute performance is low.

Integer comparison: Integer comparison questions ask which of two object sets is larger (e.g. “Are there fewer cubes than red things?”); this requires counting, memory,

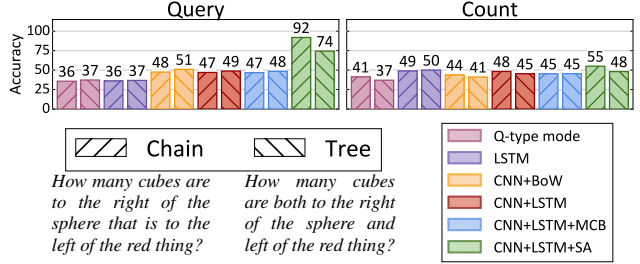


Figure 6. Accuracy on questions with two spatial relationships, broken down by question topology: chain-structured questions vs. tree-structured questions joined with a logical AND operator.

and comparing integer quantities. The answer distribution is unbiased (see Q-Type mode) but a set’s size may correlate with the length of its description, explaining the gap between LSTM and Q-type mode. CNN+BoW performs no better than chance: BoW mixes the words describing each set, making it impossible for the learner to discriminate between them. CNN+LSTM+SA outperforms LSTM on “less” and “more” questions, but no model outperforms LSTM on “equal” questions. Most models perform better on “less” than “more” due to asymmetric question families.

4.3. Analysis by Relationship Type

CLEVR questions contain two types of relationships: *spatial* and *same-attribute* (see Section 3). We can compare the relative difficulty of these two types by comparing model performance on questions with a single spatial relationship and questions with a single same-attribute relationship; results are shown in Figure 5. On query-attribute and counting questions we see that same-attribute questions are generally more difficult; the gap between CNN+LSTM+SA on spatial and same-relate query questions is particularly large (93% vs. 78%). Same-attribute relationships may require a model to keep attributes of one object “in memory” for comparison, suggesting again that models augmented with explicit memory may perform better on these questions.

4.4. Analysis by Question Topology

We next evaluate model performance on different question topologies: *chain-structured* questions vs. *tree-structured* questions with two branches joined by a logical AND (see Figure 2). In Figure 6, we compare performance on chain-structured questions with two spatial relationships vs. tree-structured questions with one relationship along each branch. On query questions, CNN+LSTM+SA shows a large gap between chain and tree questions (92% vs. 74%); on count questions, CNN+LSTM+SA slightly outperforms LSTM on chain questions (55% vs. 49%) but no method outperforms LSTM on tree questions. Tree questions may be more difficult since they require models to perform two subtasks in parallel before fusing their results.